

A comparative study of deep learning models for sentiment analysis of social media texts*

Vasily D. Derbentsev¹, Vitalii S. Bezkorovainyi¹, Andriy V. Matviychuk^{1,2},
Oksana M. Pomazun¹, Andrii V. Hrabariev¹ and Alexey M. Hostryk³

¹Kyiv National Economic University named after Vadym Hetman, 54/1 Peremogy Ave., Kyiv, 03680, Ukraine

²Kryvyi Rih State Pedagogical University, 54 Gagarin Ave., Kryvyi Rih, 50086, Ukraine

³Odessa National Economic University, 8 Preobrazhenskaya Str., Odessa, 65082, Ukraine

Abstract

Sentiment analysis is a challenging task in natural language processing, especially for social media texts, which are often informal, short, and noisy. In this paper, we present a comparative study of deep learning models for sentiment analysis of social media texts. We develop three models based on deep neural networks (DNNs): a convolutional neural network (CNN), a CNN with long short-term memory (LSTM) layers (CNN-LSTM), and a bidirectional LSTM with CNN layers (BiLSTM-CNN). We use GloVe and Word2vec word embeddings as vector representations of words. We evaluate the performance of the models on two datasets: **IMDb Movie Reviews** and **Twitter Sentiment 140**. We also compare the results with a logistic regression classifier as a baseline. The experimental results show that the CNN model achieves the best accuracy of 90.1% on the IMDb dataset, while the **BiLSTM-CNN model achieves the best accuracy of 82.1%** on the Sentiment 140 dataset. The proposed models are comparable to state-of-the-art models and suitable for practical use in sentiment analysis of social media texts.

Keywords

sentiment analysis, social media, deep learning, convolutional neural networks, long short-term memory, word embeddings

1. Introduction

The swift evolution of electronic mass media and social networks has spurred the advancement of automated Natural Language Processing (NLP) systems.

NLP resides at the crossroads of Computer Science, Artificial Intelligence, and Linguistics, dedicated to unraveling the intricacies of computer-based analysis of human language models.

The spectrum of challenges that NLP addresses is extensive. It encompasses tasks like machine

M3E2-MLPEED 2022: The 10th International Conference on Monitoring, Modeling & Management of Emergent Economy, November 17-18, 2022, Virtual, Online

*Extended and revised version of paper [1] presented at the 10th International Conference on Monitoring, Modeling & Management of Emergent Economy.

✉ derbvd@gmail.com (V. D. Derbentsev); retal.vs@gmail.com (V. S. Bezkorovainyi); editor@nfmte.com (A. V. Matviychuk); oksp@ukr.net (O. M. Pomazun); andr.grab@gmail.com (A. V. Hrabariev); alexeyGostrik@gmail.com (A. M. Hostryk)

🆔 0000-0002-8988-2526 (V. D. Derbentsev); 0000-0002-4998-8385 (V. S. Bezkorovainyi); 0000-0002-8911-5677 (A. V. Matviychuk); 0000-0001-9697-1415 (O. M. Pomazun); 0000-0001-6165-0996 (A. V. Hrabariev); 0000-0001-6143-6797 (A. M. Hostryk)



© 2023 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

CEUR Workshop Proceedings (CEUR-WS.org)

translation, speech recognition, named entity recognition, text classification and summarization, sentiment analysis, question answering, autocomplete, predictive text input, and more [2, 3, 4].

Central to NLP is Sentiment Analysis (SA), also known as opinion mining. SA endeavors to distill subjective attributes from text, such as emotions, sarcasm, confusion, and suspicion.

The crux of SA revolves around classifying the polarity of a given document, determining whether the sentiment expressed is positive, negative, or neutral.

Being a potent text classification technique, sentiment analysis can unveil a wealth of insights about viewpoints on discussed subjects. It facilitates comprehensive analysis of feedback, message polarity, and reactions. Notably, SA finds extensive utility among business professionals, marketers, and politicians.

In dissecting public sentiment regarding sensitive social and political matters, discerning prevailing themes and tonalities within discussions significantly eases the tasks of sociologists, political scientists, and journalists [5, 6].

In the face of ever-mounting information volumes, conventional methodologies have begun to falter. Swiftly monitoring and controlling public sentiment remains pivotal for success.

Historically, this challenge has been met with dictionary or rule-based approaches [7, 8, 9, 10]. These methods are statistical, relying on precompiled sentiment lexicons that pair words with respective polarities to categorize them as “positive” or “negative”.

However, constructing complete dictionaries for a large amount of unstructured data generated by modern electronic media and social networks are quite a tedious task.

Machine Learning (ML) methods [11, 12, 13] help solve this problem. Such approaches are based on algorithms for classifying words according to the corresponding sentiment marks. That’s why ML models are preferred for SA due to their ability to process with the large amount of texts compared to dictionary-based approaches.

Over the past decade, Deep Neural Networks (DNNs) have emerged as formidable tools in solving numerous NLP challenges, including SA [14, 15, 16]. This surge is underpinned by:

- Progress in crafting diverse DNN architectures (recurrent, convolutional, encoder-decoder, transformer, hybrid).
- Escalating computational prowess, bolstered by graphics processing units and a profusion of cloud computing services.
- Availability of labeled datasets tailored to various NLP tasks.
- Emergence of pre-trained word vector representations (word embeddings) like Word2Vec, FastText [17, 18, 19], extending across multiple languages.

Recent years have seen the ascendancy of colossal pre-trained models rooted in the Transformer architecture and the Attention mechanism—think GPT-3, BERT, ELMo [20, 21, 22, 23]. These models embody language models, encapsulating probability distributions across word sequences.

These models are all-encompassing, extracting features from text pivotal for solving diverse text analysis conundrums. However, they come at a computational cost—bearing hundreds of millions of parameters, necessitating formidable computational resources.

Hence, for the majority of practical NLP applications, conventional ML and Deep Learning (DL) methodologies persist as stalwarts.

Our **research aims** to architect a suite of sentiment classification models grounded in varied DNN architectures, scrutinizing their efficacy across the IMDB and Sentiment 140 Twitter datasets.

2. Related works

Drus and Khalid [24] provided a report of review on sentiment analysis in social media that explored the common methods and approaches which used in this domain. This review contains an analysis of about 30 publications published during 2014-2019 years. According to their results most of the articles applied opinion-lexicon method to analyses text sentiment in social media in such domain as world events, healthcare, politics and business.

Recently Jain et al. [25] published report on ML applications for consumer sentiment analysis in the domain of hospitality and tourism. This report based on 68 research papers, which were focused on sentiment classification, predictive recommendation decisions, and fake reviews detection.

They have shown a systematic literature review to compare, analyze, explore, and understand ML possibilities to find research gaps and the future research directions.

Sudhir and Suresh [26] published comparative study of various approaches, applications and classifiers for sentiment analysis. They have discussed the advantages and disadvantages of the different approaches such as Rule-based, ML and DL approaches used for SA as well as compared the performances of the classification models on the IMDB dataset.

The authors note that, in general, ML-based approaches provide greater accuracy than Rule-based ones. At the same time, Conventional ML models (Support Vector Machine, Decision Trees, and Logistic Regression) provide classification accuracy at the level of 85-87% for the IMDB dataset. DL-based models (CNN, LSTM, GRU) shows higher accuracy: about 89% on the IMDB dataset.

Trisna and Jie [15], presented a comparative review of DL approaches for Aspect-Based SA. The results of their analysis show that the use of pre-trained embeddings is very influential on the level of accuracy. They also found that every dataset has a different method to get better performance. It is still challenging to find the method that can be flexible and effective for using in several datasets.

There are several papers devoted to developing new methods of word embeddings.

Thus, Biesialska et al. [27] proposed a novel method which uses contextual embeddings and a self-attention mechanism to detect and classify sentiment. They performed experiments on reviews from different domains, as well as on languages (Polish and German).

Authors have shown that proposed approach is on a par with state-of-the-art models or even outperforms them in several cases.

Rasool et al. [28] proposed a novel word embedding method novel word-to-word graph (W2WG) embedding method for the real-time sentiment for word representation. He noted that performance evaluation of proposed word embedding approach with integrated LSTM-CNN outperformed the other techniques and recently available studies for the real-time sentiment classification.

Recently have been published several research papers devoted using DNNs different architecture based on CNN-LSTM models for SA task [29, 30, 31, 32, 33].

Elzayady et al. [29] presented two powerful hybrid DL models (CNN-LSTM) and (CNN-BILSTM) for reviews classification. Experimental results have shown that the two proposed models had superior performance compared to baselines DL models (CNN, LSTM).

Khan et al. [31] evaluated the performance of various word embeddings for Roman Urdu and English dialects using the CNN-LSTM architecture and compare results with traditional ML classifiers. Authors mentioned that BERT word embedding, two-layer LSTM, and SVM as a classifier function are more suitable options for English language sentiment analysis.

Priyadarshini and Cotton [32] proposed a novel LSTM-CNN grid search-based DNN model for sentiment analysis. As to the experimental results they observed proposed model performed relatively better than other algorithms (LSTM, Fully-connected NN, K-nearest neighbors, and CNN-LSTM) on Amazon reviews for sentiment analysis and IMDb datasets.

Haque et al. [33] analyzed different DNNs for SA on IMDb Movie Reviews. They have compared between CNN, LSTM and LSTM-CNN architectures for sentiment classification in order to find the best-suited architecture for this dataset. Experimental results have shown that CNN has achieved an $F1 - score$ of 91% which has outperformed LSTM, LSTM-CNN and other state-of-the-art approaches for SA on IMDb dataset.

Quraishi [34] evaluated of four ML algorithms (Multinomial Naïve Bayes, Support Vector Machine, LSTM, and GRU) for sentiment analysis on IMDb review dataset. He found that among these four algorithms, GRU performed the best with an accuracy of 89.0%.

Derbentsev et al. [35] also explored the performance of four ML algorithms (Logistic Regression, Support Vector Machine, Fully-connected NN, and CNN) for SA on IMDb dataset. They used two pre-trained word embeddings GloVe and Word2vec with different dimensions (100 and 300) as well as TF-IDF representation. They reported that the best classification accuracy (90.1%) was performed by CNN model with Word2vec-300 embedding.

3. Base concept of NLP applying to sentiment analysis

3.1. ML approach of NLP

To solve NLP problems using ML methods, it is necessary to represent the text in the form of set feature vectors. The text can consist of words, numbers, punctuation, special characters of additional markup (for example, HTML tags). Each such “unit” can be represented as a vector in various ways, for example, using unitary codes (one-hot encoding), or context-independent (depended) vector representations.

The base idea of applying ML to NLP was introduced by Bengio et al. [36]. They proposed to jointly learn an “embedding” of words into an n -dimensional numeric vector space and to use these vectors to predict how likely a word is given its context.

In the case of text, features represent attributes and properties of documents including their content and meta-attributes, such as document length, author name, source, and publication date. Together, all document features describe a multidimensional feature space to which ML methods can be applied.

Thus, in the most general terms, the application of ML to SA problems consists of the following: text data preprocessing, feature extraction, classification, and interpretation of results.

3.2. Data pre-processing

The quality of the result depends on the input data. Therefore, it is important that they are prepared in the best possible way. In general, pre-processing stage consists of the following steps [37, 38, 39]:

- *Text cleaning.* First of all, we need to clean up the text. Depending on the task, cleaning includes removing non-alphabets, various tags, URLs, punctuation, spaces, and other markup elements;
- *Segmentation and tokenization.* They are relevant in the vast majority of cases, and provide division of the text into separate sentences and words (tokens). As a rule, after tokenization all words are converted to lower case;
- *Lemmatization and stemming.* Typically, texts contain different grammatical forms of the same word, and there may also be words with the same root. Lemmatization is the process of reducing a word form to a lemma – its normal (dictionary) form. Stemming is a crude heuristic process that cuts off “excess” from the root of words, often resulting in the loss of derivational suffixes. Lemmatization is a subtler process that uses vocabulary and morphological analysis to eventually reduce a word to its canonical form, the lemma;
- *Definition of context-independent features* that characterize each of the token, which not dependent on adjacent elements;
- *Refining significance and applying a filter to stop words.* Stop words are frequently used words that do not add additional information to the text. When we apply ML to texts, such words can add a lot of noise, so it is necessary to get rid them;
- *Dependency parsing.* The result is the formation of a tree structure, where the tokens are assigned to one parent, and the type of relationship is established;
- *Converting text content to a vector representation* that highlights words used in similar or identical contexts.

3.3. Features extraction

ML algorithms cannot work directly with raw text, so it is necessary to convert the text into sets of numbers (vectors) – construct a vector representation. In ML this process is called feature extraction.

Vector representation is a general name for various approaches to language modeling and representation training in NLP aimed at matching words (and possibly phrases) from some dictionary of vectors.

The most common approaches for construction vector representations are Bag of Words, TF-IDF, and Word Embeddings [38].

3.3.1. Bag of words

Bag of words (Bow) is a popular and simple feature extraction technique used in NLP. It describes the occurrences of each word in the text.

Essentially, it creates a matrix of occurrences for a sentence or document, ignoring grammar and word order. These frequencies (“occurrences”) of words are then used as features for learning.

The basic idea of applying Bow is that similar documents have similar content. Therefore, basis on content, we can learn something about the meaning of the document.

For all its simplicity and intuitive clarity, this approach has a significant drawback. The Bow encoding uses a corpus (or set, collection) of words and represents any given text with a vector of the length of the corpus. If a word in the corpus is present in the text, the corresponding element of the vector would be the frequency of the word in the text.

If individual words are encoded by one-hot vectors, then the feature space will have a dimension equal to the cardinality of the collection’s dictionary, i.e. tens or even hundreds of thousands. This dimension rises along with the increasing of the amount of dictionary.

3.3.2. N-grams

Another, more complex way to create a dictionary is to use grouped words. This will resize the dictionary and give Bow more details about the document.

This approach is called “N-gram”. An N-gram is a sequence of any entities (words, syllable, letters, numbers, etc.). In the context of language corpora, an N-gram is usually understood as a sequence of words.

A unigram is one word, a be-gram is a sequence of two words, a trigram is three words, and so on. The number N indicates how many grouped words are included in the N-gram. Not all possible N-grams get into the model, but only those that appear in the corpus.

3.3.3. TF-IDF

Term Frequency (TF) is the ratio of the number of appearing a certain word to the total number of words in the document. Thus, the importance of a word t within a single document d_i is evaluated:

$$TF(t, d_i) = \frac{n_t}{\sum_k n_k}, \quad (1)$$

where n_t is the number of occurrences of the word t in the document d_i , and in the denominator of the fraction is the total number of words in the document.

But frequency scoring has a problem: words with the highest frequency have, accordingly, the highest score. There may not be as much information gain for the model in these words as there is in less frequent words.

One way to remedy the situation is to downgrade a word that appears frequently in all similar documents. This metric is called $TF - IDF$ (short for Term Frequency – Inverse Document Frequency).

In this metric *IDF* is the inverse of the frequency with which a certain word occurs in the documents of the collection:

$$IDF(t, d_i, D) = \log \frac{|D|}{|\{d_i \in D | t \in d_i\}|}. \quad (2)$$

Here $|D|$ is the number of documents in the collection (corpus), $\{d_i \in D | t \in d_i\}$ is the number of documents in the collection D that contain word t .

There is only one *IDF* value for each unique word within a given collection of documents. *IDF* metric reduces the weight of commonly corpusused words.

TF – IDF is a statistical measure for estimating the importance of a word in a document that is part of a collection or corpus:

$$TF-IDF(t, d_i, D) = TF(t, d_i) \times IDF(t, d_i, D). \quad (3)$$

TF – IDF scoring increases in proportion to the frequency of occurrence of the word in the document, but this is compensated by the number of documents containing this word.

The disadvantage of the frequency approach based on this metric is that it does not take into account the context of a single word. Moreover, it does not distinguish the semantic similarity of words. All vectors are equally far from each other in the feature space.

3.3.4. Word embedding

Word embedding is one of the most popular representations of document's vocabulary. This is a technique that maps words into number vectors, where words which have similar meanings will be close to each other with their vector representation in terms of some distance metric in the vector space.

Word embedding gives the impressive performance of DL methods on challenging NLP problem. Recently, several powerful word embedding models have been developed:

- Word2vec (short from Words to Vectors, provided by Google in 2013) [17];
- GloVe (short from Global Vectors, provided by Stanford University in 2014) [18];
- FastText (provided by Facebook in 2017) [19];
- BERT (short from Bidirectional Encoder Representations from Transformers, provide by Google in 2018) [40].

These models are pre-trained on large corpuses of texts, including Wikipedia and specific domain.

Word2vec is a set of ANN models designed to obtain word embedding of natural language words. It takes a large text corpus as input and maps each word to a vector, producing word coordinates as output. It first generates a dictionary of the corpus and then calculates a vector representation of the words by learning from the input texts.

The vector representation is based on contextual proximity: words that occur in the text next to the same words (and therefore have a similar meaning) will have close (by cosine distance) vectors.

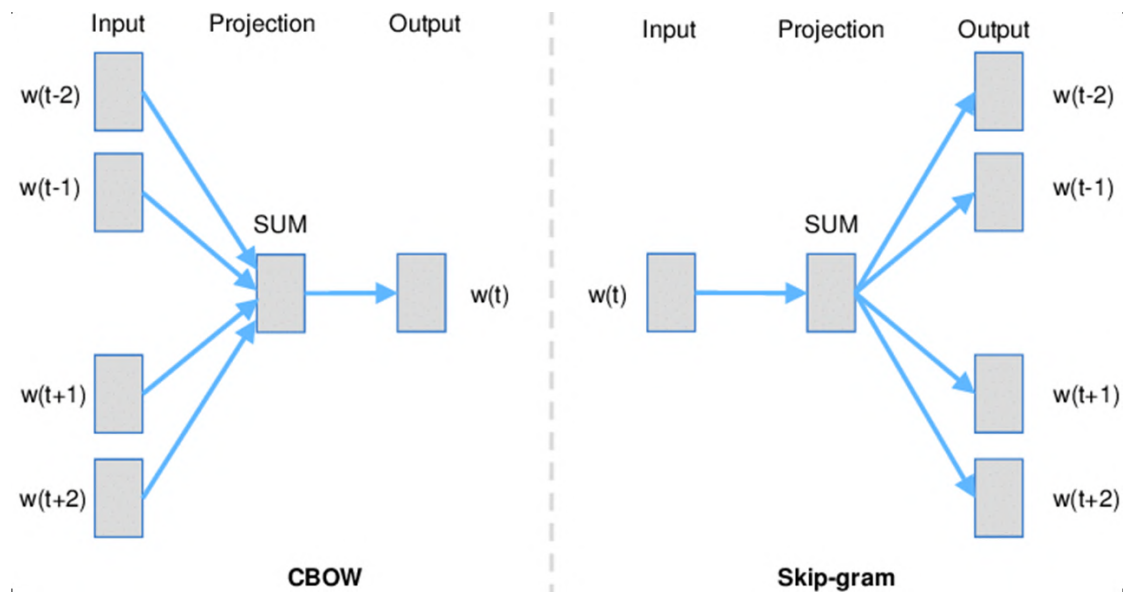


Figure 1: Simplified representation of the CBoW and Skip-gram models [17].

Word2vec implements two main learning algorithms: CBoW (Continuous Bag of Words) and Skip-gram (figure 1).

CBoW is an architecture that predicts the current word based on its surrounding context. Architecture like Skip-gram does the opposite: it uses the current word to predict surrounding words.

Building a Word2vec model is possible using these two algorithms. The word order of the context does not affect the result in any of these algorithms.

GloVe focuses on words co-occurrences over the whole corpus. Its embeddings relate to the probabilities that two words appear together. So, GloVe combines features of Word2vec and singular co-occurrence matrix decomposition.

In the present study, we applied both Word2vec and GloVe models to obtain vector representations of words.

The main application effect of using pre-trained language models is to obtain high-quality vector representations of words that take into account contextual dependencies and allow you to achieve better results on targets.

4. DNNs classification models design

After previous stage, we can start building a classification model. The model type and architecture depends on the research task of SA which can be performed at different hierarchical levels of text documents (document-level, sentence-level, word or aspect-level), domains (reviews about travel agencies, hotels, movies, election opinion prediction, analysis of public opinion on acute social and political issues), binary or multiclass classification.

If we have a dataset of texts with class labels (for example, with binary labels “positive”

and “negative”), we could apply Supervised ML techniques, in particular, binary classification algorithms.

Mathematically, this problem can be formulated as follows: given training sample of texts $X = \{x_1, x_2, \dots, x_m\}$, for each text there is a class label $Y = \{y_i\}$, $y_i \in \{0, 1\}$, $i = 1, 2, \dots, m$.

It is necessary to build a classifier model $b(X, w) : X \rightarrow Y$, where w is a vector of unknown parameters or weights.

At the same time, it is necessary to minimize the *Loss* function that determines the total deviation of real class labels from those predicted by the classifier. For binary classification problems, the most common is binary cross-entropy:

$$Loss = -\frac{1}{N} \left[\sum_{i=1}^N (y_i \log(p_i) + (1 - y_i) \log(1 - p_i)) \right] \quad (4)$$

where N is the size of the training sample, $y_i = \{0, 1\}$ is the true class label for the i -th data sample, p_i is the probability of belonging to the positive class for the i -th data sample provided by the classifier.

4.1. Logistic regression

Since the task of SA in the general case is reduced to the binary classification problem (negative, positive), we chose the Logistic Regression (LR) model as the baseline classifier $b(\cdot)$:

$$b(X, w) = \sigma(\langle w, x \rangle), \quad (5)$$

where $\langle w, x \rangle$ – denotes the scalar product, $\sigma(\cdot)$ is a *Sigmoid* (logistic) function

$$\sigma(z) = \frac{1}{1 + \exp(-z)}. \quad (6)$$

LR has such advantage as it can be used to predict the probability to belong a training sample (in our case, tokenized and vectorized text) to one of the two target classes.

4.2. CNN model

CNNs are a class of DNNs that were originally designed for image processing [41]. But these models have shown their efficiency for many other tasks, such as time series forecasting [42].

Kim [43] has shown that CNNs are efficient for classifying texts on different datasets. Recently, they have also been used for various NLP tasks (speech generation and recognition, text summarization, named entity extraction).

The architecture of CNNs consists of convolutional and subsampling layers (figure 2).

The convolutional layer performs feature extraction from the input data and generates feature maps. The feature map is computed through an element-wise multiplication of the small matrix of weights (kernel) and the matrix representation of the input data, and the result is summed.

This weighted sum then passed through the non-linear activation function. One of the most common is the function *ReLU*, which is given as $ReLU(x) = \max(0, x)$.

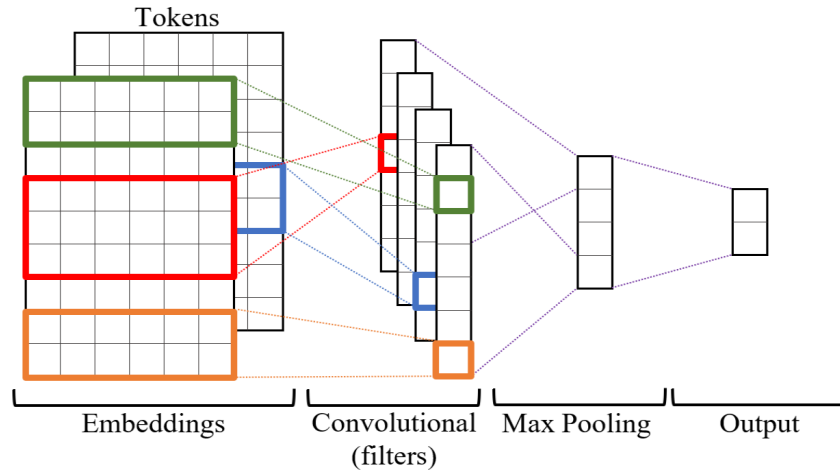


Figure 2: Convolutional Neural Network (CNN) Architecture for Text Processing [43].

The pooling (subsampling) layer is a non-linear compaction of the feature maps. For example, max-pooling takes the largest element from the feature map and extracts the sum of all its elements.

After max-pooling, feature maps are concatenated into a flattened vector, which will then be passed to a fully connected layer.

The input data for the most NLP problems is text which consists of sentences and words. So we need represent the text as an array of vectors of a certain length: each word mapped to a specific vector in a vector space composed of the entire vocabulary.

As these vectors, we can use word frequencies (for example, obtained using the $TF - IDF$ metric), or pre-trained embeddings (Word2vec, GloVe, FastText).

Unlike images processing, text convolution is performed using one-dimensional filters (1D Convolution) on one-dimensional input data, for example, sentences, using convolution kernels of different size (widths).

Applying of multiple kernels widths and feature maps is analogous to the use of N-grams.

For image processing, convolutions are usually performed on separate channels that correspond to the colors of the image: red, green, blue. Set of different filters is applied for each channel, and the result of this operation is then merged into a single vector.

For text processing as channels we can consider, for example, the sequence of words, or words embeddings. Then different kernels applied to the words can be merged into a single vector.

The final result of sentiment analysis is obtained by applying *Sigmoid* activation function (binary classification task) or *Softmax* (in the case of multi-class task).

4.3. LSTM and BiLSTM model

Sequential information and long-term dependencies in NLP traditionally performed with Recurrent Neural Networks (RNNs) which could compute context information, for example, in dependency parsing.

The most common and efficient for many ML tasks, including NLP, were architectures based on LSTM (Long Short Term Memory) or GRU (Gated Recurrent Unit) cells [37, 16].

4.3.1. LSTM

LSTM model proposed by Hochreiter and Schmidhuber [44] introduces the concept of a state for each of the layers of a RNN which plays the role of memory.

The input signal affects the state of the memory, and this, in turn, affects the output layer, just like in a RNN. But this state of memory persists throughout the time steps of a sequence (for example, time series, sentence, or text document). Therefore, each input signal affects the state of the memory as well as the output signal of the hidden layer.

LSTM cell includes several units or gates: the inputs, output, and forget gates (figure 3). These gates are used to control a memory cell that is carrying the hidden state h_t to the next time step.

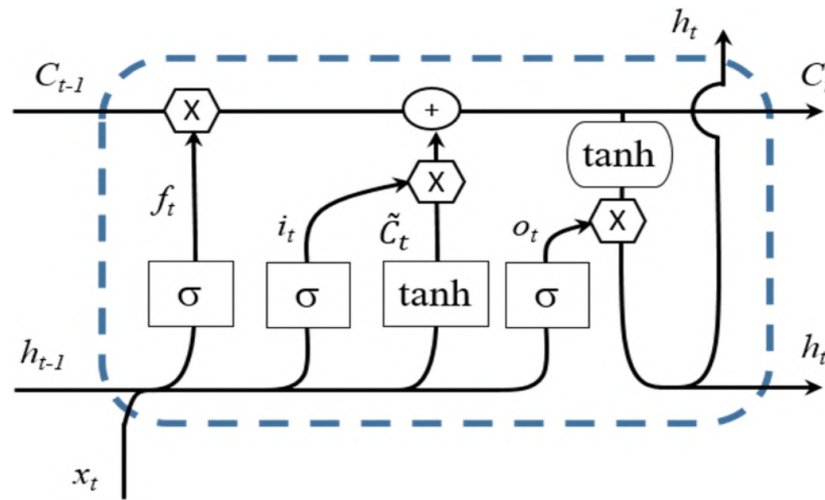


Figure 3: Diagram of a LSTM cell.

The LSTM cell is formally defined as:

$$f_t = \sigma(\mathbf{W}_f \cdot (h_{t-1}, \mathbf{x}_t) + b_f) \quad (7)$$

$$i_t = \sigma(\mathbf{W}_i \cdot (h_{t-1}, \mathbf{x}_t) + b_i) \quad (8)$$

$$\tilde{C}_t = \tanh(\mathbf{W}_c \cdot (h_{t-1}, \mathbf{x}_t) + b_c) \quad (9)$$

$$o_t = \sigma(\mathbf{W}_o \cdot (h_{t-1}, \mathbf{x}_t) + b_o) \quad (10)$$

$$a_t = i_t \otimes \tilde{C}_t \quad (11)$$

$$C_t = f_t \otimes C_{t-1} + a_t \quad (12)$$

where $\mathbf{x}_t$ – is the vector of input sequence at time t ; C_{t-1} , h_{t-1} – state (long-term content) and hidden state in previous time step ($t - 1$) respectively; $\sigma(\cdot)$, $\tanh(\cdot)$ are the *Sigmoid* and

Hyperbolic tangent activation functions; $\otimes$ – the Kronecker product; $\mathbf{W}_f, \mathbf{W}_i, \mathbf{W}_o$ – the weight matrices for input, forget, output of the gates respectively; b_f, b_i, b_o – biases for the gates.

The *input gate* i_t determines which values need to update. Then the hyperbolic tangent layer builds a vector $\hat{C}_t$ of new values that can be added to the state of the cell C_t .

The *forget gate* f_t controls how much is remembered (what part of the information is kept and what is erased) from step to step. Decision what information can be thrown out of the cell state is made by a sigmoid layer.

The *output gate* o_t receives an input signal (which is the concatenation of the input signal at time step t and the cell output signal at time step $(t - 1)$) and passes it to the output. Thus, this gate determines which part of the long-term content C_t should be transferred to the next time step.

Each of these gates is a feed-forward neural network layer consisting of a sequence of weights fitted by the network with an activation function. This allows the network to learn the conditions for forgetting, ignoring, or keeping information in the memory cell.

Due its structure LSTM can learn and remember representations for variable length sequences, such as sentences, documents, and speech samples.

4.3.2. BiLSTM

Unidirectional (standard) LSTM only preserves information of the past because the only inputs it has seen are from the past. Unlike standard LSTM, in BiLSTM (Bidirectional LSTM) model the input flows in both directions and it's capable of utilizing information from both sides.

So BiLSTM is a sequence processing model that consists of two LSTMs layers: one taking the input in a forward direction (from “past to future”), and the other in a backwards direction (from “future to past”) (figure 4).

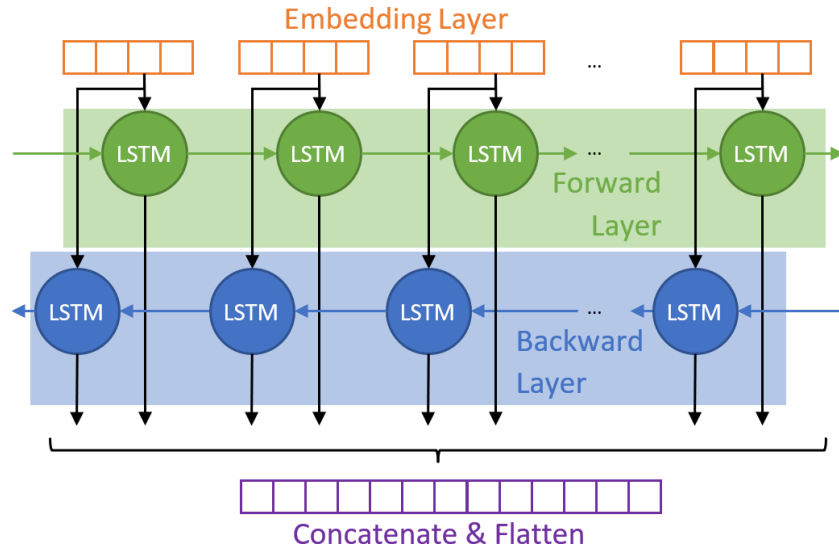


Figure 4: Diagram of a BiLSTM model.

For example, if we want to predict a word by context (the central word), the network takes a given number of words to the left of it as the context – the Forward layer performs it, as well as the words to the right of it – Backward layer performs it.

Then we can combine the outputs from both LSTM layers in different ways: as sum, average, concatenation or multiplication. This output contains the information or relation of past and future word.

BiLSTM increase the amount of information available to the network, improving the context.

It's also more powerful tool for modeling the sequential dependencies between words and phrases in both directions of the sequence than standard LSTM.

BiLSTM is usually used when we have the sequence to sequence tasks but it should be noted that BiLSTM (compared to LSTM) is a much “slower” model and requires more time for training.

4.4. CNN+LSTM model

Both basic DNNs architectures CNN and LSTM have own advantages and disadvantages. Thus, LSTM networks can capture long-term dependencies and find hidden relationships in the data. CNNs are able to extract features using different convolutions and filters.

Therefore, the combination of convolutional and recurrent layers in the model turns out to be effective in many applied problem such as simulation of various natural processes, image processing, time series forecasting, and different NLP tasks [45, 46, 47, 31, 28, 48].

So we developed two models based on modifications of CNN+LSTM architecture which final design and hyperparameters settings are given in the Section 6.

Our proposed models exploit the main features of both LSTM and CNN. In fact, LSTM could accommodate long-term dependencies and overcome the key issues with vanishing gradients. For this reason, LSTM is used when longer sequences are used as inputs. On the other hand, CNN appears able to understand local patterns and position-invariant features of a text.

5. Datasets and software implementation

All developed DNNs (CNN, CNN-LSTM, BiLSTM-CNN), and LR as the baseline, were implemented in the Python 3.8 programming language using Scikit-learn library for LR, estimation classification accuracy, and for designing DNNs models we used Keras library and TensorFlow as backend.

We evaluate the performance of our models on two datasets: Stanford's IMDB dataset (Stanford's Large Movie Review Dataset), which contains 50,000 movie reviews as well as Sentiment 140 dataset [49] with 1.6 million tweets.

Both datasets are intended for binary classification: they contain for each text (review or tweet) a sentiment class binary label. They are also balanced, i.e. contain the same number of texts for the positive and negative classes.

6. Empirical results

6.1. Pre-processing and words embeddings

For text pre-processing the Python library package NLTK [50] was used, as well as customers regular expressions.

The pre-processing stage included removing punctuations, markup tags, html and tweet addresses, removing stop words and converting all words to lower case.

Tokenization was performed by using Keras preprocessing text library. After tokenization we got the length of the vocabulary in 92393 unique tokens for IMDb dataset and 507702 for Sentiment140 respectively to which one token was added for representation out of vocabulary words.

It should be noted that the selected datasets are characterized by different average length of texts (number of words). Thus the length of most reviews does not exceed 500 words, and tweets – 50.

Since DNNs work with fixed-length input sequences we padded zero tokens all reviews and tweets which length are less than average to fixed length 500 and 50 words (tokens) respectively, and cut longer texts to these fixed sizes.

For words vector representation was used GloVe word embeddings with word vectors of dimension 100 provided by Gensim library [51].

6.2. DNNs models design and hyperparameters setting

To initialize the weights of the first layer (Embedding Layer) for all models, pre-trained GloVe embeddings of size 100 were used. These weights were frozen and did not change during training.

The first model, CNN, consists of three sequential Convolutional layers with filter sets of different kernel widths. These layers are interspersed with Maxpooling layers. Behind them are a Flatten and a Fully connected (Dense) layer.

The second, CNN-LSTM model differs from the CNN by the presence of an LSTM layer instead of a Flatten after Convolutional and Maxpooling. The base idea of such architecture is that CNN can be used to retrieve higher-level word feature sequences and LSTM to catch long-term correlations across window feature sequences, respectively.

The third, BiLSTM-CNN model contains two BiLSTM layers (forward and backward), followed by a Convolutional and Maxpooling layers. After that, two Fully connected layers were used to reduce the output dimension and make prediction.

For all models Dropout layers were also used to prevent overfitting. As the *Loss*-function Binary Cross-Entropy (4) was chosen, which can be calculated as the average cross-entropy over all data samples [52].

The final parameters of DNNs architecture are shown in table 1.

6.3. Evaluating Performance Measures

The datasets were divided in the proportion of: 64% for training, 20% for validation, and 16% for test subsets respectively.

Table 1

Final DNNs models hyperparameters setting.

Model	Layers	Parameters
CNN	Embedding	emb_dim 100, sent_len 500(50)
	Dropout	0.3
	Convolutional 1D	100 filters of size 2, act_func ReLu
	Max pooling	pool_size 2
	Convolutional 1D	100 filters of size 3, act_func ReLu
	Max pooling	pool_size 2
	Convolutional 1D	100 filters of size 4, act_func ReLu
	Max pooling	pool_size 2
	Flatten	
	Dropout	0.3
CNN-LSTM	Fully connected	1 neuron, act_func Sigmoid
	Embedding	emb_dim 100, sent_len 500(50)
	Dropout	0.3
	Convolutional 1D	50 filters of size 2, act_func ReLu
	Max pooling	pool_size 2
	Convolutional 1D	100 filters of size 2, act_func ReLu
	Max pooling	pool_size 2
	Convolutional 1D	200 filters of size 2, act_func ReLu
	Max pooling	pool_size 2
	LSTM	64 neurons, reccur_dropout 0.3
BiLSTM-CNN	Dropout	0.3
	Fully connected	32 neurons
	Fully connected	1 neuron, act_func Sigmoid
	Embedding	Emb_dim 100, sent_len 500(50)
	Dropout	0.3
	Bidirectional	LSTM with 100 neurons
	Dropout	0.3
	Bidirectional	LSTM with 100 neurons
	Dropout	0.3
	Convolutional 1D	100 filters of size 3, act_func ReLu
	Global Max pooling 1D	
	Fully connected	10 neurons, act_func ReLu
	Fully connected	1 neuron, act_func Sigmoid

All DNNs models were trained over 5 epochs with a minibatch size of 256 and 1024 samples for IMDB and Sentiment 140 respectively. To compare classification performance of the developed models we used the *Accuracy* metrics given by:

$$Accuracy = \frac{TP + TN}{P + N} \times 100\%, \quad (13)$$

where *TP* and *TN* are the number of correctly predicted values of the positive and negative classes, respectively; *P* and *N* are the actual number of values for each of the classes.

We also calculated *F1-score* which is harmonic average between *Precision* (the percentage of objects in the positive class, which were classified as positive, are correctly classified), and

Recall (percentage of objects of the true positive class which we correctly classified):

$$F1\text{-score} = \frac{2TP}{2TP + FP + FN}, \quad (14)$$

$$Precision = \frac{TP}{TP + FP}, \quad (15)$$

$$Recall = \frac{TP}{TP + FN}. \quad (16)$$

Here *FP* (False Positive) and *FN* (False Negative) are numbers of times (data samples) where the model incorrectly classified these samples as belonging to the positive and negative classes respectively.

The final results of classification performance are presented in tables 2-3.

Table 2

Classification performance on IMDB dataset, %.

Models	Precision	Recall	F1-score	Accuracy
LR (baseline)	86.62	85.54	86.08	85.90
CNN	90.04	90.31	90.18	90.09
CNN-LSTM	90.90	84.84	87.76	88.08
BiLSTM-CNN	83.08	93.25	87.87	87.03

Table 3

Classification performance on Sentiment 140 dataset, %.

Models	Precision	Recall	F1-score	Accuracy
LR (baseline)	71.61	74.63	73.09	74.23
CNN	76.17	79.47	77.78	77.24
CNN-LSTM	78.98	77.47	78.23	78.37
BiLSTM-CNN	79.54	84.41	81.91	82.10

Classification performance on IMDB dataset for all developed DNN models is better than baseline. The best *Accuracy* metric was obtained using the CNN model (90.09%). At the same time, models based on the combination of Convolutional and LSTM layers showed an *Accuracy* of 2-3% less (table 2).

It should be noted that obtained results are comparable or even superior in accuracy to the results given by other researchers [33, 34, 53] for IMDB dataset.

All models showed significantly lower accuracy (on average 10% less) on the dataset Sentiment 140 (table 3). The best result was achieved for the BiLSTM-CNN model – *Accuracy* 82.1%.

At the same time, the complication of models by adding new layers did not lead to a significant increase in accuracy, but prolonged the training time.

In our opinion, lower accuracy may be due to the fact that Sentiment 140 dataset contains many slang words that are out of vocabulary. So, if for IMDB dataset the part of the missing words was about 30 percent, then for the Sentiment 140 this part was more than 70.

7. Discussion

Our research sheds light on the effectiveness of relatively uncomplicated Deep Neural Networks (DNNs) architectures with a modest layer count for sentiment analysis of social media texts, particularly within binary classification scenarios. These models exhibit a level of accuracy that is sufficiently practical for real-world applications.

In the case of the English-language datasets, IMDb and Sentiment 140, our models showcased the following classification accuracy rates: Logistic Regression (Baseline) achieved 85.9% (74.23%), CNN achieved 90.09% (77.24%), CNN-LSTM reached 88.01% (78.36%), and BiLSTM-CNN attained 87.03% (82.10%).

Notably, preprocessing steps like lemmatization or stemming can likely boost classification accuracy. This becomes especially relevant for tweets, which frequently feature an array of user-generated vocabulary.

Another avenue for potential improvement involves utilizing word embeddings weighted by their Term Frequency-Inverse Document Frequency (TF-IDF) metric. Addressing out-of-vocabulary words could involve strategies like employing the weighted average value of neighboring word embeddings within a designated window length or substituting missing words with normalized TF-IDF embeddings transformed via principal component analysis (SVD decomposition of the sparse TF-IDF matrix to reduce dimensionality).

In our perspective, an exciting trajectory for advancing sentiment analysis in social media involves the utilization of models rooted in deep convolutional networks or the amalgamation of convolutional and recurrent networks. Coupling these models with pre-trained embeddings, such as those founded on GloVe, Word2Vec, and FastText, holds promise. Leveraging pre-trained embeddings allows the initialization of DNNs with parameters that are already somewhat attuned to the text classification task, accelerating the learning process and enhancing the generalization capabilities of classifiers founded on deep networks.

8. Conclusion

In conclusion, our research illuminates the efficacy of employing relatively straightforward Deep Neural Network (DNN) architectures for sentiment analysis in the context of social media text. Our findings underscore that even DNNs with limited complexity can yield accuracy levels suitable for practical applications in binary sentiment classification.

Through experimentation on the IMDb and Sentiment 140 datasets, we observed compelling classification accuracy results: Logistic Regression (Baseline) achieved 85.9% (74.23%), CNN achieved 90.09% (77.24%), CNN-LSTM reached 88.01% (78.36%), and BiLSTM-CNN attained 87.03% (82.10%).

Enhancing the preprocessing steps with techniques like lemmatization and incorporating weighted word embeddings via TF-IDF are potential strategies to further refine classification accuracy. Additionally, the combination of deep convolutional and recurrent networks, complemented by pre-trained embeddings, emerges as a promising avenue for advancing sentiment analysis in social media. Pre-trained embeddings not only expedite learning but also enhance the classifier's ability to generalize.

References

- [1] V. Derbentsev, V. Bezkorovainyi, A. Matviychuk, O. Pomazun, A. Hrabariev, A. Hostryk, Sentiment analysis of electronic social media based on deep learning, in: S. Semerikov, V. Soloviev, A. Matviychuk, V. Kobets, L. Kibalnyk, H. Danylchuk, A. Kiv (Eds.), Proceedings of 10th International Conference on Monitoring, Modeling & Management of Emergent Economy - M3E2, INSTICC, SciTePress, 2023, pp. 163–175. doi:10.5220/0011932300003432.
- [2] M. Azlinah, B. W. Yap, J. M. Zain, M. W. Berry (Eds.), Soft Computing in Data Science 6th International Conference, SCDS 2021, volume 1489 of *SCDS: International Conference on Soft Computing in Data Science*, Springer, Singapore, 2021. doi:10.1007/978-981-16-7334-4.
- [3] W. Mayur, C. S. R. Annavarapu, K. Chaitanya, A survey on sentiment analysis methods, applications, and challenges, *Artificial Intelligence Review* 55 (2022) 5731–5780. doi:10.1007/s10462-022-10144-1.
- [4] M. Silberztein, F. Atigui, E. Kornysheva, E. Métais, F. Meziane (Eds.), Natural Language Processing and Information Systems: Proceedings of 23rd International Conference on Applications of Natural Language to Information Systems, volume 10859 of *Lecture Notes in Computer Science*, Springer, Cham, 2018. doi:10.1007/978-3-319-91947-8.
- [5] C. A. Iglesias, A. Moreno (Eds.), Sentiment Analysis for Social Media, MDPI, 2020. doi:10.3390/books978-3-03928-573-0.
- [6] F. Pozzi, E. Fersini, E. Messina, B. Liu, Sentiment Analysis in Social Networks, Elsevier Science, 2016.
- [7] H. Karamollaoğlu, İ. A. Doğru, M. Dörterler, A. Utku, O. Yıldız, Sentiment Analysis on Turkish Social Media Shares through Lexicon Based Approach, in: 2018 3rd International Conference on Computer Science and Engineering (UBMK), 2018, pp. 45–49. URL: <https://ieeexplore.ieee.org/document/8566481>.
- [8] C. Dhaoui, C. M. Webster, L. P. Tan, Social media sentiment analysis: lexicon versus machine learning, *Journal of Consumer Marketing* 34 (2017) 480–488. doi:10.1108/JCM-03-2017-2141.
- [9] C. S. Khoo, S. B. Johnkhan, Lexicon-based sentiment analysis: Comparative evaluation of six sentiment lexicons, *Journal of Information Science* 44 (2018) 491–511. doi:10.1177/0165551517703514.
- [10] D. Alessia, F. Ferri, P. Grifoni, T. Guzzo, Approaches, tools and applications for sentiment analysis implementation, *International Journal of Computer Applications* 125 (2015) 26–33.
- [11] A. Kiv, S. Semerikov, V. N. Soloviev, L. Kibalnyk, H. Danylchuk, A. Matviychuk, Experimental Economics and Machine Learning for Prediction of Emergent Economy Dynamics, in: A. Kiv, S. Semerikov, V. N. Soloviev, L. Kibalnyk, H. Danylchuk, A. Matviychuk (Eds.), Proceedings of the Selected Papers of the 8th International Conference on Monitoring, Modeling & Management of Emergent Economy, M3E2-EEMLPEED 2019, Odessa, Ukraine, May 22–24, 2019, volume 2422 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2019, pp. 1–4. URL: <https://ceur-ws.org/Vol-2422/paper00.pdf>.
- [12] V. Derbentsev, A. Matviychuk, V. N. Soloviev, Forecasting of Cryptocurrency Prices Using Machine Learning, in: L. Pichl, C. Eom, E. Scalas, T. Kaizoji (Eds.), Advanced Studies of Financial Technologies and Cryptocurrency Markets, Springer, Singapore, 2020, pp.

- 211–231. doi:10.1007/978-981-15-4498-9_12.
- [13] P. V. Zahorodko, S. O. Semerikov, V. N. Soloviev, A. M. Striuk, M. I. Striuk, H. M. Shalatska, Comparisons of performance between quantum-enhanced and classical machine learning algorithms on the IBM Quantum Experience, *Journal of Physics: Conference Series* 1840 (2021) 012021. doi:10.1088/1742-6596/1840/1/012021.
 - [14] H. Li, Deep learning for natural language processing: advantages and challenges, *National Science Review* 5 (2017) 24–26. doi:10.1093/nsr/nwx110.
 - [15] K. W. Trisna, H. J. Jie, Deep Learning Approach for Aspect-Based Sentiment Classification: A Comparative Review, *Applied Artificial Intelligence* 36 (2022) 2014186. doi:10.1080/08839514.2021.2014186.
 - [16] U. Kamath, J. Liu, J. Whitaker, *Deep Learning for NLP and Speech Recognition*, Springer, Cham, 2019. doi:10.1007/978-3-030-14596-5.
 - [17] T. Mikolov, K. Chen, G. Corrado, J. Dean, Efficient Estimation of Word Representations in Vector Space, in: Y. Bengio, Y. LeCun (Eds.), *1st International Conference on Learning Representations, ICLR 2013, Scottsdale, Arizona, USA, May 2-4, 2013, Workshop Track Proceedings*, 2013. URL: <https://arxiv.org/abs/1301.3781>.
 - [18] J. Pennington, R. Socher, C. Manning, GloVe: Global Vectors for Word Representation, in: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, Doha, Qatar, 2014, pp. 1532–1543. doi:10.3115/v1/D14-1162.
 - [19] P. Bojanowski, E. Grave, A. Joulin, T. Mikolov, Enriching Word Vectors with Subword Information, *Transactions of the Association for Computational Linguistics* 5 (2017) 135–146. doi:10.1162/tac1_a_00051.
 - [20] A. K. Durairaj, A. Chinnalagu, Transformer based Contextual Model for Sentiment Analysis of Customer Reviews: A Fine-tuned BERT, *International Journal of Advanced Computer Science and Applications* 12 (2021). doi:10.14569/IJACSA.2021.0121153.
 - [21] M. P. Geetha, D. Karthika Renuka, Improving the performance of aspect based sentiment analysis using fine-tuned Bert Base Uncased model, *International Journal of Intelligent Networks* 2 (2021) 64–69. doi:10.1016/j.ijin.2021.06.005.
 - [22] H. Deng, D. Ergu, F. Liu, Y. Cai, B. Ma, Text sentiment analysis of fusion model based on attention mechanism, *Procedia Computer Science* 199 (2022) 741–748. doi:10.1016/j.procs.2022.01.092.
 - [23] S. Tabinda Kokab, S. Asghar, S. Naz, Transformer-based deep learning models for the sentiment analysis of social media data, *Array* 14 (2022) 100157. doi:<https://doi.org/10.1016/j.array.2022.100157>.
 - [24] Z. Drus, H. Khalid, Sentiment Analysis in Social Media and Its Application: Systematic Literature Review, *Procedia Computer Science* 161 (2019) 707–714. doi:10.1016/j.procs.2019.11.174.
 - [25] P. K. Jain, R. Pamula, G. Srivastava, A systematic literature review on machine learning applications for consumer sentiment analysis using online reviews, *Computer Science Review* 41 (2021) 100413. doi:10.1016/j.cosrev.2021.100413.
 - [26] P. Sudhir, V. D. Suresh, Comparative study of various approaches, applications and classifiers for sentiment analysis, *Global Transitions Proceedings* 2 (2021) 205–211. doi:<https://doi.org/10.1016/j.gltp.2021.08.004>.

- [27] M. Biesialska, K. Biesialska, H. Rybinski, Leveraging contextual embeddings and self-attention neural networks with bi-attention for sentiment analysis, *Journal of Intelligent Information Systems* 57 (2021) 601–626. doi:10.1007/s10844-021-00664-7.
- [28] A. Rasool, Q. Jiang, Q. Qu, C. Ji, WRS: A Novel Word-embedding Method for Real-time Sentiment with Integrated LSTM-CNN Model, in: 2021 IEEE International Conference on Real-time Computing and Robotics (RCAR), 2021, pp. 590–595. doi:10.1109/RCAR52367.2021.9517671.
- [29] H. Elzayady, M. S. Mohamed, S. Badran, Integrated bidirectional LSTM-CNN model for customers reviews classification, *Journal of Engineering Science and Military Technologies* 5 (2021). doi:10.21608/EJMTC.2021.66626.1172.
- [30] N. Hernández, I. Batyrshin, G. Sidorov, Evaluation of deep learning models for sentiment analysis, *Journal of Intelligent & Fuzzy Systems* (2022) 1–11. doi:10.3233/JIFS-211909.
- [31] L. Khan, A. Amjad, K. M. Afaq, H.-T. Chang, Deep Sentiment Analysis Using CNN-LSTM Architecture of English and Roman Urdu Text Shared in Social Media, *Applied Sciences* 12 (2022) 2694. doi:10.3390/app12052694.
- [32] I. Priyadarshini, C. Cotton, A novel LSTM-CNN-grid search-based deep neural network for sentiment analysis, *The Journal of Supercomputing* 77 (2021) 13911–13932. doi:10.1007/s11227-021-03838-w.
- [33] M. R. Haque, H. Salma, S. A. Lima, S. M. Zaman, Performance Analysis of Different Neural Networks for Sentiment Analysis on IMDb Movie Reviews, 2020. URL: <https://www.researchgate.net/publication/343046458>.
- [34] A. H. Quraishi, Performance Analysis of Machine Learning Algorithms for Movie Review, *International Journal of Computer Applications* 177 (2020) 7–10. doi:10.5120/ijca2020919839.
- [35] V. Derbentsev, V. Bezkorovainyi, R. Akhmedov, Machine Learning Approach of Analysis Emotion Polarity Electronic Social Media, *Neiro-Nechitki Tekhnolohii Modelyuvannya v Ekonomitsi* 9 (2020).
- [36] Y. Bengio, R. Ducharme, P. Vincent, C. Jauvin, A neural probabilistic language model, *Journal of Machine Learning Research* 3 (2003) 1137–1155. URL: <https://proceedings.neurips.cc/paper/2000/file/728f206c2a01bf572b5940d7d9a8fa4c-Paper.pdf>.
- [37] J. Brownlee, Develop Deep Learning Models for Natural Language in Python. Deep Learning for Natural Language Processing, 2017. URL: http://ling.snu.ac.kr/class/AI_Agent/deep_learning_for_nlp.pdf.
- [38] L. Hobson, H. Cole, H. Hannes, Natural Language Processing in Action: Understanding, analyzing, and generating text with Python, Manning Publications, 2019.
- [39] J. Camacho-Collados, M. T. Pilehvar, On the Role of Text Preprocessing in Neural Network Architectures: An Evaluation Study on Text Categorization and Sentiment Analysis, in: Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP, Association for Computational Linguistics, Brussels, Belgium, 2018, pp. 40–46. doi:10.18653/v1/W18-5406.
- [40] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, 2018. URL: <https://arxiv.org/abs/1810.04805>.
- [41] Y. LeCun, Y. Bengio, Convolutional Networks for Images, Speech, and Time Series, in: The Handbook of Brain Theory and Neural Networks, MIT Press, Cambridge, MA, USA,

- 1998, p. 255–258.
- [42] Y. LeCun, Y. Bengio, G. Hinton, Deep learning, *Nature* 521 (2015) 436–444.
 - [43] Y. Kim, Convolutional Neural Networks for Sentence Classification, in: A. Moschitti, B. Pang, W. Daelemans (Eds.), *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25-29, 2014, Doha, Qatar, A meeting of SIGDAT, a Special Interest Group of the ACL, ACL, 2014*, pp. 1746–1751. doi:10.3115/v1/d14-1181.
 - [44] S. Hochreiter, J. Schmidhuber, Long Short-Term Memory, *Neural Computation* 9 (1997) 1735–1780. doi:10.1162/neco.1997.9.8.1735.
 - [45] N. Chen, P. Wang, Advanced Combined LSTM-CNN Model for Twitter Sentiment Analysis, in: *2018 5th IEEE International Conference on Cloud Computing and Intelligence Systems (CCIS)*, 2018, pp. 684–687. doi:10.1109/CCIS.2018.8691381.
 - [46] V. Derbentsev, V. Bezkorovainyi, M. Silchenko, A. Hrabariev, O. Pomazun, Deep Learning Approach for Short-Term Forecasting Trend Movement of Stock Indices, in: *2021 IEEE 8th International Conference on Problems of Infocommunications, Science and Technology (PIC S&T)*, 2021, pp. 607–612. doi:10.1109/PICST54195.2021.9772235.
 - [47] M. Z. Islam, M. M. Islam, A. Asraf, A combined deep CNN-LSTM network for the detection of novel coronavirus (COVID-19) using X-ray images, *Informatics in Medicine Unlocked* 20 (2020) 100412. doi:10.1016/j.imu.2020.100412.
 - [48] L. Shang, L. Sui, S. Wang, D. Zhang, Sentiment analysis of film reviews based on CNN-BLSTM-attention, *Journal of Physics: Conference Series* 1550 (2020) 032056. doi:10.1088/1742-6596/1550/3/032056.
 - [49] Kaggle, Sentiment140 dataset with 1.6 million tweets, 2022. URL: <https://www.kaggle.com/datasets/kazanova/sentiment140>.
 - [50] NLTK Project, Natural language toolkit, 2022. URL: <https://www.nltk.org/>.
 - [51] R. Řehůřek, Gensim: Topic modelling for humans, 2022. URL: <https://radimrehurek.com/gensim/>.
 - [52] A. Geron, *Hands-On Machine Learning with Scikit-Learn and TensorFlow*, O'Reilly Media, Inc., 2017.
 - [53] N. M. Ali, M. M. A. El Hamid, A. Youssif, Sentiment Analysis for Movies Reviews Dataset Using Deep Learning Models, *International Journal of Data Mining & Knowledge Management Process (IJDKP)* 9 (2019). URL: <https://aircconline.com/abstract/ijdkp/v9n3/9319ijdkp02.html>.